

Modèles de langage et LLM

Fondements, adaptation, RAG, agents, évaluation et production

Synthèse

2026

Table des matières

Mode d'emploi	9
1 NLP et modèles de langage	10
1.1 Du texte aux tâches	10
1.2 Qu'est-ce qu'un modèle de langage ?	10
1.3 Maximum de vraisemblance et perplexité	10
1.4 Corpus et unités de données	11
1.5 Normalisation du texte	11
2 Tokenisation et représentations vectorielles	12
2.1 Pourquoi tokeniser ?	12
2.2 Mots, caractères et sous-mots	12
2.3 Longueur et coût	12
2.4 Embeddings	13
2.5 Similarité	13
2.6 Word2Vec	13
3 RNN, LSTM et limites des modèles séquentiels	14
3.1 Réseau récurrent	14
3.2 Backpropagation Through Time	14
3.3 LSTM	14
3.4 Seq2Seq et attention	15
3.5 Limites	15
4 Attention et architecture Transformer	16
4.1 Pourquoi l'attention ?	16
4.2 Query, Key et Value	16

4.3	Exemple de poids	16
4.4	Multi-Head Attention	17
4.5	Information positionnelle	17
4.6	Bloc Transformer	17
4.7	Masque causal	18
4.8	Encodeur et décodeur	18
4.9	Complexité	18
4.10	KV Cache	18
4.11	Exercice corrigé	18
5	Pré-entraînement des modèles de langage	20
5.1	Principe	20
5.2	Causal Language Modeling	20
5.3	Masked Language Modeling	20
5.4	Denoising et span corruption	21
5.5	Données	21
5.6	Échelle et compute	21
5.7	Optimisation distribuée	21
5.8	Précision numérique	21
5.9	Validation du pré-entraînement	21
5.10	Contamination et mémorisation	22
5.11	Pré-entraînement continu	22
6	Familles BERT, GPT et encodeur-décodeur	23
6.1	Encodeur-only	23
6.2	Décodeur-only	23
6.3	Encodeur-décodeur	23
6.4	Comparaison	24
6.5	Classification avec un encodeur	24
6.6	Extraction d'entités	24
6.7	Instruction tuning	24
6.8	Préférences	24
6.9	Choix pratique	25

7	Contexte, génération et décodage	26
7.1	Fenêtre de contexte	26
7.2	Préfill et décodage	26
7.3	Distribution du prochain token	26
7.4	Greedy et beam search	27
7.5	Top-k et nucleus sampling	27
7.6	Pénalités et arrêt	27
7.7	Teacher forcing et exposition	27
7.8	Logprob et confiance	27
7.9	Décodage contraint	28
8	Prompting et sorties structurées	29
8.1	Anatomie d'un prompt	29
8.2	Zero-shot et few-shot	29
8.3	Décomposition	29
8.4	Raisonnement et réponses	29
8.5	Sorties structurées	30
8.6	Séparer instructions et données	30
8.7	Prompt injection	30
8.8	Templates et versioning	30
8.9	Optimisation empirique	30
8.10	Exemple d'extraction	31
8.11	Tests	31
9	Fine-tuning, instruction tuning et PEFT	32
9.1	Quand adapter le modèle ?	32
9.2	Supervised Fine-Tuning	32
9.3	Qualité des données	32
9.4	Full Fine-Tuning	33
9.5	LoRA	33
9.6	Choix LoRA	33
9.7	QLoRA	33
9.8	Autres PEFT	33

9.9	Préférences : DPO	33
9.10	Évaluation de l'adaptation	34
9.11	Pipeline pratique	34
10	Embeddings, recherche vectorielle et RAG	35
10.1	Pourquoi augmenter un LLM?	35
10.2	Architecture générale	35
10.3	Embeddings de passages	35
10.4	Chunking	36
10.5	Recherche lexicale et dense	36
10.6	Approximate Nearest Neighbors	36
10.7	Filtrage	36
10.8	Reranking	36
10.9	Reformulation et décomposition	37
10.10	Construction du contexte	37
10.11	Génération grounded	37
10.12	Évaluation séparée	37
10.13	Diagnostic RAG	37
10.14	Mise à jour et suppression	38
10.15	RAG ou fine-tuning?	38
11	Tool calling et agents	39
11.1	Du texte à l'action	39
11.2	Schéma d'outil	39
11.3	Boucle agentique	39
11.4	État et mémoire	40
11.5	Planification	40
11.6	ReAct	40
11.7	Validation des arguments	40
11.8	Moindre privilège	40
11.9	Idempotence et reprise	40
11.10	Conditions d'arrêt	41
11.11	Human-in-the-loop	41

11.12	Agents multiples	41
11.13	Évaluation	41
11.14	Observabilité	41
11.15	Sécurité contre l'injection	41
12	Évaluation, hallucinations et sécurité	43
12.1	Pourquoi l'évaluation est difficile	43
12.2	Construire un jeu d'évaluation	43
12.3	Métriques déterministes	43
12.4	Préférences et comparaisons par paires	44
12.5	LLM comme juge	44
12.6	Incertitude	44
12.7	Hallucinations	44
12.8	Mesurer factuelité et fidélité	44
12.9	Réduire les hallucinations	45
12.10	Calibration et abstention	45
12.11	Taxonomie des risques	45
12.12	Injection de prompt	45
12.13	Protection des données	45
12.14	Biais et équité	46
12.15	Red teaming	46
12.16	Défense en profondeur	46
13	Déploiement, optimisation et monitoring	47
13.1	Du prototype au service	47
13.2	Préfill et décodage	47
13.3	Batching continu	47
13.4	Cache KV	48
13.5	Quantification	48
13.6	Distillation et pruning	48
13.7	Décodage spéculatif	48
13.8	Choisir le modèle	48
13.9	Cache applicatif	49

13.10	Coût	49
13.11	SLO et capacité	49
13.12	Observabilité	49
13.13	Versionnement et livraison progressive	49
13.14	Dérive et boucle de retour	50
13.15	Incidents	50
13.16	Architecture de référence	50
I	Mise en pratique	51
14	TP 1 – Construire un petit Transformer	52
14.1	Objectifs	52
14.2	Données et protocole	52
14.3	Dimensions à vérifier	52
14.4	Étapes	52
14.5	Expériences	53
14.6	Questions d’analyse	53
15	TP 2 – Adapter un modèle avec LoRA	54
15.1	Objectifs	54
15.2	Préparer les exemples	54
15.3	Principe	54
15.4	Protocole	54
15.5	Analyse d’erreurs	55
15.6	Livrables	55
16	TP 3 – Construire et évaluer un RAG	56
16.1	Objectifs et corpus	56
16.2	Pipeline minimal	56
16.3	Baseline lexicale et dense	56
16.4	Jeu d’évaluation	56
16.5	Expériences	57
16.6	Rapport	57

17 TP 4 – Agent outillé sous contraintes	58
17.1 Scénario	58
17.2 Contrats d’outils	58
17.3 Contrôleur	58
17.4 Tests	58
17.5 Discussion	59
 II Exercices et corrigés	 60
18 Exercices	61
18.1 Modélisation du langage	61
18.2 Tokenisation et embeddings	61
18.3 Attention	61
18.4 Entraînement et adaptation	61
18.5 Génération	62
18.6 RAG et agents	62
18.7 Évaluation et production	62
 19 Corrigés	 63
19.1 Modélisation du langage	63
19.2 Tokenisation et embeddings	63
19.3 Attention	63
19.4 Entraînement et adaptation	63
19.5 Génération	64
19.6 RAG et agents	64
19.7 Évaluation et production	64
 III Révision et références	 65
20 Aide-mémoire mathématique	66
20.1 Probabilités et information	66
20.2 Algèbre et attention	66
20.3 Optimisation	66

20.4 Évaluation	66
21 Glossaire	68
22 Fiches de révision	69
Fiche 1 – Modèles de langage	70
Fiche 2 – Tokens et vecteurs	71
Fiche 3 – RNN et LSTM	72
Fiche 4 – Transformer	73
Fiche 5 – Pré-entraînement	74
Fiche 6 – Familles de modèles	75
Fiche 7 – Génération	76
Fiche 8 – Prompting	77
Fiche 9 – Adaptation	78
Fiche 10 – RAG	79
Fiche 11 – Agents	80
Fiche 12 – Évaluation et sécurité	81
Fiche 13 – Production	82
23 Études de cas transversales	83
23.1 Assistant documentaire interne	83
23.2 Extraction de contrats	83
23.3 Support client outillé	83
23.4 Synthèse médicale assistée	84
23.5 Assistant de programmation	84
23.6 Choisir l’architecture	84
24 Bibliographie commentée	85
Conclusion générale	87

Mode d'emploi

Ce cours distingue les principes durables des outils susceptibles d'évoluer. Les mathématiques expliquent les mécanismes ; les exemples relient théorie et pratique ; les encadrés signalent décisions et pièges.

Chapitre 1

NLP et modèles de langage

1.1 Du texte aux tâches

Le traitement automatique du langage transforme du texte en sorties utiles : classe, séquence, représentation, réponse ou action. Les tâches principales incluent classification, extraction d'entités, recherche, traduction, résumé, question-réponse et génération.

Le texte possède ambiguïté, dépendances longues, polysémie et structure contextuelle. Une même chaîne peut changer de sens selon la phrase ; des formulations différentes peuvent exprimer une intention proche. Le système doit aussi distinguer connaissance linguistique, connaissance du monde et information fournie dans le contexte.

1.2 Qu'est-ce qu'un modèle de langage ?

Pour une séquence de tokens $x_{1:T}$, un modèle autorégressif factorise

$$P(x_{1:T}) = \prod_{t=1}^T P(x_t \mid x_{<t}).$$

Il apprend à prédire le prochain token, puis génère récursivement. Un modèle masqué estime plutôt certains tokens à partir du contexte gauche et droit. L'objectif d'apprentissage ne garantit ni vérité factuelle ni respect d'une intention humaine : il optimise une distribution de texte.

1.3 Maximum de vraisemblance et perplexité

La négative log-vraisemblance moyenne est

$$\mathcal{L}_{NLL} = -\frac{1}{T} \sum_{t=1}^T \log P_{\theta}(x_t \mid x_{<t}).$$

La perplexité est

$$PPL = \exp(\mathcal{L}_{NLL}).$$

Une perplexité plus faible indique une meilleure prédiction sur le corpus considéré, mais ne mesure pas directement utilité, factualité, sécurité ou capacité à suivre une consigne.

1.4 Corpus et unités de données

Un corpus peut contenir documents, dialogues, code ou données spécialisées. Il faut documenter source, licence, langue, date, doublons et informations personnelles. Le découpage Train/Validation/Test doit éviter la contamination par documents identiques ou quasi identiques.

Attention. Une bonne performance sur un benchmark contaminé mesure potentiellement la mémorisation. Dédupliquer et rechercher les recouvrements entre pré-entraînement, adaptation et évaluation.

1.5 Normalisation du texte

Les opérations possibles incluent normalisation Unicode, espaces, casse, caractères de contrôle et segmentation. Une normalisation agressive peut détruire accents, code, mise en page ou indices utiles. Les modèles modernes requièrent surtout le prétraitement exact associé à leur tokenizer.

Chapitre 2

Tokenisation et représentations vectorielles

2.1 Pourquoi tokeniser ?

Un réseau manipule des indices, pas directement des chaînes. Un tokenizer définit une fonction

$$\mathcal{T} : \text{texte} \rightarrow (t_1, \dots, t_n), \quad t_i \in \{0, \dots, V - 1\}.$$

Le vocabulaire V doit équilibrer couverture, longueur des séquences et capacité à représenter mots rares, langues et code.

2.2 Mots, caractères et sous-mots

Une tokenisation par mots produit un vocabulaire immense et des mots inconnus. Les caractères évitent l'inconnu mais allongent fortement les séquences. Les sous-mots offrent un compromis : mots fréquents entiers, mots rares décomposés. BPE fusionne itérativement les paires fréquentes ; WordPiece choisit des unités utiles au modèle ; Unigram part d'un grand vocabulaire et le réduit probabilistiquement.

2.3 Longueur et coût

Le nombre de tokens diffère du nombre de mots et varie selon la langue. Pour une attention dense, le stockage de la matrice d'attention croît comme

$$O(n^2),$$

où n est la longueur tokenisée. Le tokenizer influence donc coût, latence et quantité de contexte utilisable.

2.4 Embeddings

Une matrice

$$E \in \mathbb{R}^{V \times d}$$

associe au token t le vecteur $\mathbf{e}_t = E[t]$. Des embeddings statiques donnent un vecteur unique par type ; des représentations contextuelles donnent un vecteur dépendant de la phrase.

2.5 Similarité

La similarité cosinus est

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2}.$$

Elle compare l'orientation mais pas la norme. Pour une recherche, le choix cosinus, produit scalaire ou distance euclidienne doit correspondre à l'entraînement et à la normalisation du modèle.

2.6 Word2Vec

Skip-gram prédit le contexte à partir du mot central. Avec negative sampling, un couple positif (w, c) et des contextes négatifs n_i donnent

$$\mathcal{L} = -\log \sigma(\mathbf{v}_c^\top \mathbf{v}_w) - \sum_i \log \sigma(-\mathbf{v}_{n_i}^\top \mathbf{v}_w).$$

Ces embeddings capturent des régularités distributionnelles, mais un mot polysémique conserve une seule représentation.

Réflexe. Avant d'utiliser des embeddings, préciser l'unité représentée, le modèle, la normalisation, la métrique, la langue, le domaine et la taille maximale du texte.

Chapitre 3

RNN, LSTM et limites des modèles séquentiels

3.1 Réseau récurrent

Un RNN met à jour un état caché :

$$\mathbf{h}_t = \tanh(W_x \mathbf{x}_t + W_h \mathbf{h}_{t-1} + \mathbf{b}), \quad \mathbf{y}_t = W_y \mathbf{h}_t + \mathbf{c}.$$

Les mêmes poids sont partagés dans le temps. Le réseau traite naturellement une longueur variable, mais son calcul est séquentiel et difficile à paralléliser.

3.2 Backpropagation Through Time

Le réseau déroulé applique la règle de chaîne sur de nombreux pas. Les produits répétés de Jacobiennes peuvent faire disparaître ou exploser le gradient. Gradient clipping limite la norme ; LSTM et GRU introduisent des chemins de mémoire contrôlés par des portes.

3.3 LSTM

Une écriture classique est

$$\mathbf{f}_t = \sigma(W_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f), \quad \mathbf{i}_t = \sigma(W_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i),$$

$$\tilde{\mathbf{c}}_t = \tanh(W_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c),$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t,$$

$$\mathbf{o}_t = \sigma(W_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o), \quad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t).$$

La porte d'oubli conserve ou supprime l'ancien état ; la porte d'entrée écrit une information ; la porte de sortie contrôle l'état visible.

3.4 Seq2Seq et attention

Un encodeur résume la source et un décodeur génère la cible. Un unique vecteur devient un goulot d'étranglement pour les longues séquences. L'attention permet au décodeur de consulter tous les états encodeurs :

$$\alpha_{t,s} = \frac{\exp(e_{t,s})}{\sum_j \exp(e_{t,j})}, \quad \mathbf{c}_t = \sum_s \alpha_{t,s} \mathbf{h}_s.$$

3.5 Limites

Le calcul récurrent empêche un parallélisme complet pendant l'entraînement. Les dépendances longues restent difficiles et l'information traverse de nombreux pas. Le Transformer remplacera la récurrence par l'attention comme mécanisme principal.

Réflexe. RNN/LSTM restent utiles pour comprendre l'histoire et certains systèmes contraints, mais pour un nouveau grand modèle de langage, le Transformer constitue généralement le point de départ.

Chapitre 4

Attention et architecture Transformer

4.1 Pourquoi l'attention ?

Un modèle doit relier des tokens éloignés et sélectionner l'information pertinente. Contrairement à un RNN, l'attention crée un chemin direct entre toutes les positions et permet de traiter les tokens en parallèle pendant l'entraînement.

4.2 Query, Key et Value

Pour une matrice d'entrées $X \in \mathbb{R}^{n \times d}$:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

Une query décrit ce qu'une position recherche ; une key ce qu'une position peut offrir ; une value l'information transmise. L'attention produit

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V.$$

Le facteur $\sqrt{d_k}$ empêche les produits scalaires de devenir trop grands. Le masque M vaut zéro pour les relations permises et une grande valeur négative pour celles qui doivent être interdites.

4.3 Exemple de poids

Pour la position i , les scores sont

$$s_{ij} = \frac{q_i^\top k_j}{\sqrt{d_k}}, \quad \alpha_{ij} = \frac{e^{s_{ij}}}{\sum_l e^{s_{il}}},$$

puis

$$o_i = \sum_j \alpha_{ij} v_j.$$

Les coefficients sont positifs et leur somme vaut un. Ils représentent une pondération interne, mais ne constituent pas automatiquement une explication fidèle de la décision finale.

4.4 Multi-Head Attention

Chaque tête travaille dans un sous-espace :

$$head_h = \text{Attention}(XW_h^Q, XW_h^K, XW_h^V),$$

$$\text{MHA}(X) = \text{Concat}(head_1, \dots, head_H)W_O.$$

Plusieurs têtes permettent d'apprendre différentes relations. Leur spécialisation n'est toutefois ni garantie ni toujours interprétable.

4.5 Information positionnelle

Sans position, l'attention est équivariante aux permutations. On ajoute donc une information de position. L'encodage sinusoïdal historique est

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right),$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d}}\right).$$

D'autres méthodes emploient positions apprises, biais relatifs ou rotations comme RoPE. Les positions relatives facilitent souvent la représentation des distances et l'extrapolation, mais le comportement au-delà du contexte d'entraînement doit être vérifié.

4.6 Bloc Transformer

Un bloc contient attention, connexion résiduelle, normalisation et MLP. Dans une variante pre-norm :

$$H' = H + \text{MHA}(\text{LN}(H)),$$

$$H'' = H' + \text{MLP}(\text{LN}(H')).$$

Le MLP positionnel utilise typiquement deux projections et une activation :

$$\text{MLP}(x) = W_2 \phi(W_1 x + b_1) + b_2.$$

Les résidus créent des chemins courts pour l'information et le gradient ; LayerNorm stabilise l'échelle par token.

4.7 Masque causal

Dans un décodeur autorégressif, la position t ne doit pas lire le futur :

$$M_{t,j} = \begin{cases} 0 & j \leq t, \\ -\infty & j > t. \end{cases}$$

Le softmax attribue alors un poids nul aux positions futures. Pendant l'entraînement, tous les prochains tokens peuvent être prédits en parallèle grâce à ce masque ; pendant la génération, les tokens sont produits séquentiellement.

4.8 Encodeur et décodeur

Un encodeur utilise une attention bidirectionnelle et produit une représentation contextuelle. Un décodeur emploie une attention causale. Dans un Transformer encodeur-décodeur, le décodeur ajoute une cross-attention vers les sorties de l'encodeur :

$$Q = H_{dec}W_Q, \quad K = H_{enc}W_K, \quad V = H_{enc}W_V.$$

4.9 Complexité

L'attention dense construit une matrice $n \times n$ et coûte approximativement $O(n^2d)$ en temps, avec une mémoire quadratique pour les scores. Les longs contextes motivent attention locale, sparse, récurrente, approximation ou kernels optimisés. Augmenter la fenêtre ne garantit pas une utilisation uniforme de toute l'information.

4.10 KV Cache

À l'inférence autorégressive, les keys et values des tokens précédents peuvent être mémorisées. Sans cache, le préfixe serait recalculé à chaque pas. La mémoire du cache croît approximativement avec

$$O(Lnd),$$

où L est le nombre de couches. Le cache accélère la génération mais devient une contrainte majeure pour les longs contextes et les grands batches.

4.11 Exercice corrigé

Avec $n = 2048$, la matrice d'attention contient $n^2 = 4\,194\,304$ scores par tête et par exemple. Doubler n multiplie ce nombre par quatre. Ce calcul explique pourquoi longueur, batch et mémoire interagissent fortement.

Réflexe. Pour lire un Transformer, identifier : tokenizer, dimension d , nombre de couches, nombre de têtes, type de positions, masque, taille du contexte, architecture encodeur/décodeur et objectif d'entraînement.

Chapitre 5

Pré-entraînement des modèles de langage

5.1 Principe

Le pré-entraînement apprend des régularités à partir d'un grand corpus avant l'adaptation à une tâche. La performance dépend autant des données, de l'objectif et du calcul que de l'architecture.

5.2 Causal Language Modeling

L'objectif autorégressif minimise

$$\mathcal{L}_{CLM} = - \sum_{t=1}^T \log P_{\theta}(x_t \mid x_{<t}).$$

Il correspond naturellement à la génération. Le décalage des labels est crucial : la sortie à la position t prédit le token $t + 1$.

5.3 Masked Language Modeling

Un sous-ensemble $\mathcal{M}$ de tokens est masqué :

$$\mathcal{L}_{MLM} = - \sum_{t \in \mathcal{M}} \log P_{\theta}(x_t \mid x_{\setminus \mathcal{M}}).$$

Le modèle utilise les contextes gauche et droit. Il convient aux représentations et tâches de compréhension, mais n'est pas directement un générateur causal standard.

5.4 Denoising et span corruption

Un encodeur-décodeur peut reconstruire un texte à partir d'une version corrompue. Masquer des spans force l'apprentissage de dépendances plus larges qu'un masquage token isolé. Cet objectif relie pré-entraînement et génération conditionnelle.

5.5 Données

Le pipeline comprend collecte, filtrage, déduplication, détection de langue, suppression de secrets et données personnelles, contrôle de licence, tokenisation et mélange des sources. Le mélange est une décision de modélisation : surreprésenter une source modifie compétences et biais.

5.6 Échelle et compute

Paramètres N , nombre de tokens D et calcul C doivent être équilibrés. Un modèle surdimensionné entraîné sur trop peu de tokens peut être moins efficace qu'un modèle plus petit mieux entraîné. Les lois d'échelle sont empiriques et dépendent du régime, des données et de l'architecture.

5.7 Optimisation distribuée

Le data parallelism répartit les batches ; le tensor parallelism répartit les opérations d'une couche ; le pipeline parallelism répartit les couches. Le sharding distribue paramètres, gradients et états d'optimiseur. Ces stratégies échangent mémoire contre communication et complexité.

5.8 Précision numérique

FP16 et BF16 réduisent mémoire et accélèrent le calcul. BF16 offre une plage d'exposants plus large ; FP16 peut nécessiter loss scaling. Les poids maîtres, gradients et réductions peuvent employer des précisions différentes.

5.9 Validation du pré-entraînement

Suivre loss et perplexité par source, stabilité des gradients, débit, consommation, exemples générés et évaluations intermédiaires. Une baisse globale peut masquer une dégradation sur une langue rare. Un checkpoint corrompu ou une divergence coûte cher : sauvegarde et reprise doivent être testées.

5.10 Contamination et mémorisation

Dédupliquer les données et rechercher les recouvrements avec les benchmarks. La mémorisation est favorisée par répétition et rareté. Les tests peuvent inclure détection de chaînes verbatim, canaries et audits de données, tout en respectant la confidentialité.

5.11 Pré-entraînement continu

Continuer l'entraînement sur un domaine spécialisé adapte vocabulaire et connaissances, mais peut provoquer oubli ou déplacement de comportement. Mélanger des données générales, réduire le learning rate et comparer avant/après sur plusieurs domaines.

Attention. Pré-entraînement plus récent ne signifie pas automatiquement connaissance fiable ou à jour. Les données ont une date de coupure, peuvent se contredire et ne fournissent aucune garantie de vérité.

Chapitre 6

Familles BERT, GPT et encodeur-décodeur

6.1 Encodeur-only

Un modèle de type BERT utilise l'attention bidirectionnelle. Chaque token est représenté avec son contexte complet. Il est adapté à classification, NER, recherche dense et extraction. Pour classifier, on utilise un token agrégé ou un pooling :

$$z = Wh_{pool} + b.$$

6.2 Décodeur-only

Un modèle de type GPT utilise un masque causal et optimise le prochain token. Il est naturellement adapté à complétion, dialogue, code et génération conditionnée par une instruction. Toutes les informations – système, consigne, exemples, documents et outils – sont sérialisées dans le contexte selon un format de chat.

6.3 Encodeur-décodeur

Des modèles comme T5 représentent les tâches sous forme texte-vers-texte. L'encodeur lit toute la source ; le décodeur génère la cible avec self-attention causale et cross-attention. Cette famille est naturelle pour traduction, résumé et transformations conditionnelles.

6.4 Comparaison

	Encodeur	Décodeur	Encodeur-décodeur
Contexte	Bidirectionnel	Causal	Source bidirectionnelle, cible causale
Force	Compréhension	Génération	Transformation conditionnelle
Sortie	Représentation	Suite de tokens	Suite conditionnée
Exemples	BERT	GPT	T5/BART

6.5 Classification avec un encodeur

On tokenise, ajoute padding et masque d'attention, puis entraîne une tête. Pour K classes :

$$\mathcal{L} = -\frac{1}{N} \sum_i \log P(y_i | x_i).$$

Le padding mask empêche les tokens artificiels de contribuer. Une troncature mal choisie peut supprimer l'information décisive.

6.6 Extraction d'entités

Chaque token reçoit une classe. La tokenisation en sous-mots exige un alignement entre labels de mots et tokens. On peut labelliser seulement le premier sous-token ou propager le label selon le protocole. Les métriques doivent être calculées au niveau entité, pas uniquement token.

6.7 Instruction tuning

Le pré-entraînement causal apprend à continuer du texte. L'instruction tuning entraîne sur des couples consigne-réponse afin de rendre l'interface plus utile. La loss est souvent masquée sur la partie prompt pour optimiser surtout la réponse, selon le format choisi.

6.8 Préférences

Une adaptation aux préférences apprend à favoriser des réponses jugées meilleures. Une loss paire simple peut prendre la forme

$$\mathcal{L} = -\log \sigma(r_\theta(x, y^+) - r_\theta(x, y^-)).$$

Les méthodes peuvent entraîner un reward model puis optimiser une policy, ou comparer directement réponses préférée et rejetée. Les préférences reflètent annotateurs, consignes et compromis ; elles ne sont pas une vérité universelle.

6.9 Choix pratique

Pour embeddings ou classification à grande échelle, un encodeur compact est souvent efficace. Pour interaction et génération générale, un décodeur instruction-tuned est naturel. Pour une transformation structurée avec source longue et cible courte, un encodeur-décodeur peut être avantageux. Le choix final dépend qualité, latence, mémoire, licence et langue.

Réflexe. Ne pas demander “quel est le meilleur LLM ?” sans préciser tâche, langue, contexte, budget, latence, confidentialité et méthode d’évaluation.

Chapitre 7

Contexte, génération et décodage

7.1 Fenêtre de contexte

Le contexte contient instructions, historique, exemples, documents et sorties d'outils. Sa longueur est mesurée en tokens. Si l'entrée occupe n_{in} tokens et la sortie réservée n_{out} , il faut respecter

$$n_{in} + n_{out} \leq n_{context}.$$

Une grande fenêtre ne garantit pas que chaque détail soit utilisé. L'information peut être diluée, contradictoire ou située dans une zone moins bien exploitée.

7.2 Préfill et décodage

Le préfill traite le prompt en parallèle et construit le KV cache. Le décodage produit ensuite les tokens un par un. On distingue donc la latence avant le premier token, le débit de génération et la latence totale. Ces mesures réagissent différemment à la longueur du prompt et de la réponse.

7.3 Distribution du prochain token

À l'étape t , le modèle produit des logits $z \in \mathbb{R}^V$. Avec une température $T > 0$:

$$p_i(T) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}.$$

Si $T < 1$, la distribution devient plus concentrée ; si $T > 1$, elle devient plus plate. La température ne crée pas de connaissance et ne corrige pas une mauvaise consigne.

7.4 Greedy et beam search

Le greedy choisit $\arg \max_i p_i$ à chaque étape : rapide et déterministe, mais local. Beam search conserve plusieurs préfixes et maximise approximativement

$$\log P(y_{1:T} \mid x) = \sum_t \log P(y_t \mid y_{<t}, x).$$

Une pénalité de longueur évite de favoriser systématiquement les séquences courtes. Beam search convient à certaines tâches contraintes, mais peut produire un texte répétitif en génération ouverte.

7.5 Top-k et nucleus sampling

Top- k limite l'échantillonnage aux k tokens les plus probables. Top- p choisit le plus petit ensemble S tel que

$$\sum_{i \in S} p_i \geq p.$$

Top- p adapte le nombre de candidats à l'incertitude locale. Pour extraction et JSON, une faible stochasticité est préférable ; pour idéation, une diversité contrôlée peut être utile.

7.6 Pénalités et arrêt

Les pénalités de répétition modifient les logits des tokens déjà utilisés. Elles peuvent réduire les boucles mais aussi altérer noms, code ou termes récurrents légitimes. Les stop sequences doivent être choisies sans tronquer une sortie valide. Une limite de tokens protège coût et latence mais peut couper la réponse.

7.7 Teacher forcing et exposition

Pendant l'entraînement causal, le modèle reçoit le vrai préfixe. À l'inférence, il reçoit ses propres sorties. Une erreur peut donc modifier tout le futur : c'est l'exposure bias. Les évaluations doivent mesurer la séquence générée, pas seulement la loss token par token.

7.8 Logprob et confiance

La logprob d'une séquence est une somme dépendant de sa longueur. Elle ne fournit pas directement une probabilité de véracité. Comparer des choix peut exiger normalisation et calibration sur la tâche. Pour une décision critique, prévoir abstention, validation externe ou humain.

7.9 Décodage contraint

Une grammaire ou un schéma peut interdire les tokens incompatibles avec une structure. Cela améliore la validité syntaxique d'un JSON, mais pas la vérité sémantique de ses champs. Validation de schéma, règles métier et gestion des erreurs restent nécessaires.

Réflexe. Choisir le décodage selon le produit : déterministe pour extraction, échantillonné pour créativité, contraint pour structure. Mesurer qualité, taux de validité, latence et coût.

Chapitre 8

Prompting et sorties structurées

8.1 Anatomie d'un prompt

Un prompt robuste précise rôle, objectif, données, contraintes, format et critères de réussite. Séparer clairement instructions et contenu non fiable. Une structure utile est : contexte, tâche, règles, entrées délimitées, format de sortie et exemples.

8.2 Zero-shot et few-shot

Zero-shot fournit seulement la consigne. Few-shot ajoute des démonstrations (x_i, y_i) dans le contexte. Les exemples enseignent format et décision, mais consomment des tokens et peuvent introduire biais d'ordre ou analogies trompeuses. Choisir des cas représentatifs, limites et erreurs fréquentes.

8.3 Décomposition

Une tâche complexe peut être séparée en sous-tâches vérifiables : extraire, normaliser, calculer, contrôler puis rédiger. La décomposition réduit parfois les erreurs et facilite les tests, mais multiplie appels, latence et points de panne.

8.4 Raisonnement et réponses

Demander une justification concise ou des étapes vérifiables peut aider, mais une explication générée n'est pas une trace garantie du mécanisme interne. Pour les calculs et faits, privilégier outils, contraintes et vérifications plutôt qu'une longue chaîne verbale invérifiable.

8.5 Sorties structurées

Pour un objet de sortie Y , définir un schéma avec types, champs obligatoires, enums et contraintes. Exemple :

```
{
  "sentiment": "positif|neutre|negatif",
  "score": 0.0,
  "evidence": ["court extrait"]
}
```

Après génération : parser, valider le schéma, contrôler les règles métier et réessayer de manière bornée. Ne pas exécuter directement une chaîne produite par le modèle.

8.6 Séparer instructions et données

Un document récupéré, un email ou une page web est une donnée potentiellement hostile. Le délimiter et préciser qu'il ne modifie pas les règles. Cette précaution réduit sans éliminer la prompt injection ; les permissions d'outils doivent fournir la vraie barrière.

8.7 Prompt injection

Une injection cherche à faire suivre au modèle des instructions présentes dans les données. Défenses : moindre privilège, allowlists d'actions, confirmation pour opérations sensibles, isolation des secrets, validation des arguments et journalisation. Un "ignore les instructions" dans le prompt n'est pas une frontière de sécurité.

8.8 Templates et versioning

Un prompt de production est un artefact versionné. Enregistrer modèle, paramètres de génération, template, schéma et exemples. Comparer les versions sur un jeu d'évaluation fixe. Modifier simultanément modèle et prompt empêche d'attribuer le gain.

8.9 Optimisation empirique

Construire d'abord une baseline simple, collecter les erreurs, puis ajouter seulement les instructions qui corrigent des modes d'échec observés. Un prompt très long peut contenir contradictions et règles oubliées. Préférer des contraintes nettes et testables.

8.10 Exemple d'extraction

Une bonne consigne indique la taxonomie, la gestion de l'absence, le format et l'interdiction d'inventer. Les cas ambigus peuvent produire un champ `needs_review`. Le score déclaré par le modèle doit être calibré avant d'être interprété comme probabilité.

8.11 Tests

Inclure cas normaux, limites, entrées vides, longues, multilingues, contradictoires, malveillantes et hors domaine. Mesurer exactitude champ par champ, validité JSON, refus correct, latence et stabilité entre répétitions.

Attention. Un prompt ne remplace ni contrôle d'accès, ni validation de schéma, ni sandbox. Le modèle doit être traité comme un composant probabiliste produisant des données non fiables.

Chapitre 9

Fine-tuning, instruction tuning et PEFT

9.1 Quand adapter le modèle ?

Le prompting suffit souvent pour format ou comportement simple. Le RAG apporte une connaissance externe. Le fine-tuning est pertinent pour style stable, taxonomie, format, jargon ou comportement répété. Il est mauvais comme base de données fréquemment mise à jour.

9.2 Supervised Fine-Tuning

Avec des exemples consigne-réponse, on minimise

$$\mathcal{L}_{SFT} = - \sum_{t \in \mathcal{R}} \log P_{\theta}(y_t \mid x, y_{<t}),$$

où $\mathcal{R}$ désigne souvent les tokens de réponse. Le format de chat, les tokens spéciaux et le masque de loss doivent correspondre au tokenizer et au modèle.

9.3 Qualité des données

Quelques exemples cohérents peuvent valoir mieux qu'un grand corpus bruité. Dédupliquer, équilibrer les tâches, supprimer secrets et réponses incorrectes. Séparer les jeux d'entraînement, de validation et de test par source ou groupe. Une réponse cible doit représenter le comportement réellement souhaité.

9.4 Full Fine-Tuning

Tous les paramètres sont modifiés. Cette méthode offre une capacité maximale mais demande mémoire pour poids, gradients et états d'optimiseur, et augmente le risque d'oubli. Elle est justifiée avec assez de données, calcul et besoin d'adaptation profonde.

9.5 LoRA

Pour une matrice gelée $W \in \mathbb{R}^{d_{out} \times d_{in}}$, LoRA apprend une mise à jour basse-rang :

$$W' = W + \Delta W, \quad \Delta W = \frac{\alpha}{r} B A,$$

avec $A \in \mathbb{R}^{r \times d_{in}}$, $B \in \mathbb{R}^{d_{out} \times r}$ et $r \ll \min(d_{in}, d_{out})$. Le nombre de paramètres entraînés devient $r(d_{in} + d_{out})$ au lieu de $d_{in}d_{out}$.

9.6 Choix LoRA

On cible souvent projections d'attention et parfois MLP. Le rang r contrôle la capacité ; α l'échelle ; dropout régularise. Davantage de modules ou un rang élevé augmentent mémoire et capacité, sans garantir un meilleur Test.

9.7 QLoRA

QLoRA conserve le modèle de base quantifié et entraîne des adaptateurs en précision supérieure. Cela réduit fortement la mémoire, mais ajoute des compromis numériques et de débit. La compatibilité matériel-bibliothèque et la qualité après fusion ou chargement doivent être testées.

9.8 Autres PEFT

Adapters ajoutent de petits modules ; prefix/prompt tuning apprend des vecteurs continus ; BitFit ajuste surtout les biais. Le choix dépend taille des données, nombre de tâches, coût de stockage et architecture.

9.9 Préférences : DPO

Pour une réponse préférée y_w , rejetée y_l , une policy π_θ et une référence π_{ref} , DPO optimise une préférence relative, schématiquement

$$-\log \sigma \left(\beta \left[\log \frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)} - \log \frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)} \right] \right).$$

La qualité dépend fortement des paires, du désaccord des annotateurs et du choix de référence.

9.10 Évaluation de l'adaptation

Comparer base, prompt optimisé et modèle adapté sur le même Test. Mesurer tâche cible, capacités générales, sécurité, format, calibration et latence. Une amélioration moyenne peut masquer un oubli sur une langue ou un sous-domaine.

9.11 Pipeline pratique

```
Definir comportement -> auditer donnees -> split par groupe  
-> baseline prompting -> SFT/LoRA -> eval pendant entraînement  
-> checkpoint -> test final -> tests adversariaux -> déploiement
```

Réflexe. Utiliser prompting pour instruire, RAG pour fournir des faits, fine-tuning pour modifier durablement le comportement. Ces leviers peuvent être combinés mais résolvent des problèmes différents.

Chapitre 10

Embeddings, recherche vectorielle et RAG

10.1 Pourquoi augmenter un LLM ?

Les paramètres d'un modèle ne constituent pas une base de connaissances fiable, traçable ou immédiatement actualisable. Le Retrieval-Augmented Generation récupère des sources au moment de la requête puis les fournit dans le contexte. Il vise factualité, fraîcheur et attribution, sans garantir automatiquement une réponse correcte.

10.2 Architecture générale

```
Documents -> parsing -> nettoyage -> chunks -> embeddings ->
index
Question -> reformulation -> embedding -> recherche -> reranking
-> contexte -> LLM -> reponse avec sources
```

Le pipeline possède deux phases : indexation hors ligne et interrogation en ligne. Chaque étape peut introduire une erreur distincte.

10.3 Embeddings de passages

Un encodeur produit $e(x) \in \mathbb{R}^d$. Après normalisation $\|e(x)\|_2 = 1$, produit scalaire et cosinus coïncident :

$$e(q)^\top e(d) = \cos(e(q), e(d)).$$

Un modèle d'embedding doit être évalué sur langue, domaine, longueur et type de recherche. Mélanger une indexation avec un modèle et les requêtes avec un autre invalide généralement la géométrie.

10.4 Chunking

Un chunk trop court perd le contexte ; trop long dilue le passage utile et consomme la fenêtre. Les stratégies incluent longueur fixe avec overlap, découpage par titres, paragraphes, fonctions de code ou unités sémantiques. Conserver titre, page, date, source et permissions comme métadonnées.

10.5 Recherche lexicale et dense

BM25 favorise les termes discriminants. Une forme de score est

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1(1 - b + b|d|/\text{avgdl})}.$$

La recherche dense capture mieux paraphrases et proximité sémantique. La recherche hybride combine les deux, par exemple par fusion de rangs réciproques :

$$\text{RRF}(d) = \sum_s \frac{1}{k + \text{rank}_s(d)}.$$

10.6 Approximate Nearest Neighbors

Une recherche exacte devient coûteuse avec des millions de vecteurs. Les index ANN échangent rappel, latence et mémoire. HNSW construit un graphe navigable ; des méthodes de partitionnement et quantification réduisent mémoire. Mesurer recall@k de l'index par rapport à une recherche exacte sur un échantillon.

10.7 Filtrage

Les métadonnées limitent la recherche par utilisateur, langue, date, produit ou version. Le contrôle d'accès doit être appliqué avant de transmettre les passages au modèle. Une instruction dans le prompt ne remplace pas le filtrage d'autorisation.

10.8 Reranking

Le retriever récupère rapidement un ensemble large ; un reranker plus coûteux réordonne les candidats. Un cross-encoder lit conjointement requête et passage et estime une pertinence plus fine. Le schéma fréquent est top-50 par retrieval, puis top-5 après reranking.

10.9 Reformulation et décomposition

Une question conversationnelle peut dépendre de l'historique. Une reformulation autonome facilite la recherche, mais peut altérer l'intention. Les questions multi-hop peuvent être décomposées en plusieurs recherches. Conserver la requête originale pour contrôle et évaluation.

10.10 Construction du contexte

Ordonner les passages, enlever les doublons, respecter le budget et conserver les citations. Les sources récupérées sont des données non fiables pouvant contenir des instructions malveillantes. Délimiter les documents et interdire leur influence sur les permissions.

10.11 Génération grounded

La consigne demande de répondre uniquement à partir des sources, de citer et d'indiquer lorsque l'information manque. Cela réduit les hallucinations sans les éliminer. Une citation doit réellement soutenir la proposition associée ; la simple présence d'une URL n'est pas une attribution correcte.

10.12 Évaluation séparée

Le retrieval est évalué avec recall@k , precision@k , MRR ou nDCG. Pour une requête q avec premier résultat pertinent au rang r_q :

$$MRR = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{r_q}.$$

La génération est évaluée sur exactitude, complétude, fidélité aux sources, citations, refus lorsque les sources sont insuffisantes et style.

10.13 Diagnostic RAG

Si la réponse est fausse, déterminer : document absent du corpus ; parsing incorrect ; mauvais chunk ; passage non récupéré ; reranking défaillant ; passage présent mais ignoré ; synthèse erronée. Sans cette attribution, changer le LLM peut masquer la cause.

10.14 Mise à jour et suppression

Versionner documents et embeddings. Une modification doit réindexer les chunks concernés ; une suppression doit disparaître de l’index et des caches. Les citations doivent pointer vers la version réellement utilisée.

10.15 RAG ou fine-tuning ?

RAG convient aux faits changeants, privés et citables. Le fine-tuning convient au comportement, format, style et compétences répétées. Pour une expertise spécialisée, on peut combiner adaptation du modèle et RAG sur les sources actuelles.

Réflexe. Évaluer d’abord si le bon passage apparaît dans le top-k. Si non, corriger ingestion et retrieval ; si oui mais la réponse reste fausse, corriger contexte, prompt ou générateur.

Chapitre 11

Tool calling et agents

11.1 Du texte à l'action

Un outil expose une opération structurée : rechercher, calculer, lire une base, envoyer ou modifier. Le modèle choisit un outil et propose des arguments ; l'application valide, exécute puis renvoie le résultat. Le modèle ne doit pas exécuter directement du code ou des commandes non contrôlés.

11.2 Schéma d'outil

Un bon schéma possède nom explicite, description précise, types stricts, champs obligatoires, enums et contraintes. Exemple :

```
{
  "name": "get_weather",
  "arguments": {
    "city": "Lyon",
    "date": "2026-09-08"
  }
}
```

Des outils redondants ou ambigus augmentent les erreurs de sélection.

11.3 Boucle agentique

```
Objectif -> observer -> planifier -> appeler un outil
          -> lire le resultat -> verifier -> continuer ou repondre
```

Un agent est donc un système combinant modèle, état, outils, politique de contrôle et conditions d'arrêt. Le LLM seul n'est pas l'agent complet.

11.4 État et mémoire

La mémoire de travail contient le contexte de la tâche. Une mémoire persistante stocke seulement des informations explicitement utiles et autorisées. Résumer peut perdre contraintes et provenance ; conserver des données personnelles inutilement augmente le risque.

11.5 Planification

Pour une tâche longue, décomposer en étapes et vérifier les préconditions. Replanifier lorsqu'un résultat contredit les hypothèses. Un plan généré n'est pas une autorisation : les actions sensibles restent gouvernées par des règles externes.

11.6 ReAct

Le principe alterne raisonnement opérationnel et actions observables. En production, les traces internes détaillées ne sont pas nécessaires ; conserver plutôt décisions, appels, arguments, résultats et justifications courtes auditables.

11.7 Validation des arguments

Avant exécution : parser le JSON, valider types, bornes, identifiants, autorisations et préconditions. Après exécution : traiter erreurs, sorties vides, timeout et données hostiles. Les résultats d'outils sont des entrées non fiables.

11.8 Moindre privilège

Chaque outil ne reçoit que les permissions nécessaires. Séparer lecture et écriture. Pour supprimer, payer, publier ou envoyer, exiger une autorisation ou confirmation adaptée. Les secrets ne doivent jamais être placés dans le prompt si le modèle n'en a pas besoin.

11.9 Idempotence et reprise

Une opération idempotente peut être répétée sans effet supplémentaire. Pour les écritures externes, utiliser clés d'idempotence, journal d'état et vérification avant retry. Après un timeout, ne pas supposer que l'action a échoué : vérifier son état.

11.10 Conditions d'arrêt

Limiter nombre d'étapes, coût, durée et répétitions. Détecter boucle, absence de progrès et appels identiques. En cas de blocage, l'agent doit demander une information ou restituer un état clair plutôt qu'inventer.

11.11 Human-in-the-loop

L'humain peut approuver actions irréversibles, résoudre ambiguïtés, examiner faibles confiances et arbitrer les conflits. L'escalade doit transmettre objectif, éléments vérifiés, action proposée et conséquences.

11.12 Agents multiples

Plusieurs agents peuvent spécialiser recherche, code ou critique, mais ajoutent communication, coût et incohérences. Une orchestration parallèle est utile pour des sous-tâches réellement indépendantes. Un évaluateur automatique ne garantit pas que le résultat soit vrai.

11.13 Évaluation

Mesurer réussite de tâche, choix d'outil, exactitude des arguments, nombre d'étapes, coût, latence, violations de politique, récupération après erreur et besoin d'intervention humaine. Rejouer des scénarios déterministes avec outils simulés facilite les tests.

11.14 Observabilité

Journaliser version du modèle, prompt, outils disponibles, appels, durées, erreurs et résultat final, en masquant secrets et données sensibles. Utiliser des trace IDs pour reconstruire un parcours. Une bonne observabilité permet de distinguer erreur de raisonnement, outil ou intégration.

11.15 Sécurité contre l'injection

Une page peut demander à l'agent d'exfiltrer un secret. Les données ne peuvent pas accorder de permissions. Défenses : isolation des secrets, allowlists, sandbox, filtrage des destinations, validation et confirmation. Le modèle n'est jamais l'unique contrôle de sécurité.

Attention. Plus d'autonomie augmente la surface d'erreur. Commencer par un workflow borné et déterministe ; ajouter planification et mémoire seulement lorsqu'elles apportent un gain mesuré.

Chapitre 12

Évaluation, hallucinations et sécurité

12.1 Pourquoi l'évaluation est difficile

Un LLM peut produire plusieurs bonnes réponses à une même question. La qualité est multidimensionnelle : exactitude, pertinence, fidélité aux sources, style, sécurité, latence et coût. Une moyenne unique masque souvent des échecs importants. Il faut donc relier chaque mesure à un usage et à un risque.

12.2 Construire un jeu d'évaluation

Le jeu doit représenter les entrées réelles : cas courants, cas limites, formulations ambiguës, langues, documents longs et tentatives adversariales. On sépare développement et test final pour éviter d'optimiser sur les exemples de référence. Les données doivent être versionnées avec leur provenance, leurs droits d'usage et des critères de succès explicites.

Une stratification par difficulté ou catégorie rend le diagnostic plus utile qu'un score global. Si n_k exemples appartiennent à la strate k , une moyenne pondérée est

$$S = \sum_{k=1}^K w_k S_k, \quad \sum_{k=1}^K w_k = 1.$$

Les poids peuvent refléter la fréquence en production ou la gravité métier.

12.3 Métriques déterministes

Pour une extraction structurée, on mesure validité du format, exact match, précision, rappel et F1 :

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F_1 = 2 \frac{PR}{P + R}.$$

ROUGE ou BLEU comparent des recouvrements de texte mais pénalisent des paraphrases correctes. Une similarité d'embeddings tolère mieux la reformulation, sans garantir la

vérité.

12.4 Préférences et comparaisons par paires

Pour les tâches ouvertes, un évaluateur compare deux réponses sans connaître leur origine. Le taux de victoire de A contre B vaut

$$W_A = \frac{N_{A>B} + \frac{1}{2}N_{A=B}}{N}.$$

L'ordre des réponses doit être alterné pour réduire le biais de position. Les annotateurs disposent d'une grille et d'exemples d'ancrage ; leur accord est lui-même mesuré.

12.5 LLM comme juge

Un modèle juge permet de tester rapidement beaucoup de cas, mais il peut préférer les réponses longues, son propre style ou des arguments convaincants mais faux. On calibre le juge sur un sous-ensemble annoté par des humains, on randomise l'ordre et on exige un verdict structuré fondé sur des critères. Pour les décisions à fort enjeu, il reste un signal, pas une autorité.

12.6 Incertitude

Un score observé sur un échantillon varie. Le bootstrap consiste à rééchantillonner les n exemples avec remise, recalculer le score et utiliser les quantiles de la distribution obtenue comme intervalle. Une différence minuscule sans intervalle de confiance ne justifie pas nécessairement un changement de modèle.

12.7 Hallucinations

Une hallucination est une information non étayée ou contradictoire avec la réalité ou les sources imposées. On distingue hallucination intrinsèque, qui contredit le contexte, et extrinsèque, qui ajoute une affirmation invérifiable. Une réponse peut être fluide et fausse ; la probabilité linguistique n'est pas une probabilité de vérité.

12.8 Mesurer factuelité et fidélité

Décomposer la réponse en affirmations atomiques, puis vérifier chacune contre une source autorisée. La fidélité peut être résumée par

$$\text{Fidélité} = \frac{\text{affirmations soutenues}}{\text{affirmations vérifiables}}.$$

Il faut aussi mesurer la complétude : une réponse qui ne dit presque rien peut être fidèle mais inutile. Pour un RAG, évaluer séparément récupération, contexte et génération localise la panne.

12.9 Réduire les hallucinations

Clarifier le domaine, fournir des sources, demander des citations vérifiables et autoriser explicitement “je ne sais pas”. Utiliser des outils déterministes pour calculs et données fraîches. Vérifier après génération les champs sensibles. Une température basse augmente la reproductibilité mais ne transforme pas une connaissance absente en fait correct.

12.10 Calibration et abstention

Le système doit savoir différer. Une politique peut répondre seulement si un score c dépasse un seuil τ . On analyse alors compromis couverture–risque : augmenter τ réduit les réponses mais améliore souvent leur fiabilité. La confiance verbalisée par le modèle doit être calibrée empiriquement, jamais prise au pied de la lettre.

12.11 Taxonomie des risques

Les risques incluent contenu dangereux, discrimination, atteinte à la vie privée, fuite de secrets, désinformation, automatisation abusive et conseils à fort enjeu. Leur importance dépend de la capacité du système : un chatbot informatif n’a pas le même impact qu’un agent capable d’effectuer un virement.

12.12 Injection de prompt

Une instruction hostile peut se trouver dans une page, un courriel ou un document récupéré. Elle cherche à remplacer l’objectif, révéler des données ou déclencher un outil. Séparer instructions de confiance et données, limiter les permissions, filtrer les destinations et faire confirmer les actions sensibles. Aucun texte lu ne peut conférer une nouvelle autorisation.

12.13 Protection des données

Minimiser les données envoyées, masquer les identifiants inutiles, définir rétention et localisation, chiffrer transit et stockage, et contrôler les journaux. Un historique utile pour le débogage peut devenir une base de données sensible. Les utilisateurs doivent savoir quelles informations sont mémorisées et pouvoir les corriger ou les supprimer.

12.14 Biais et équité

Évaluer les performances par sous-groupe lorsque cela est légal, pertinent et respectueux de la vie privée. Un score moyen identique peut cacher des écarts. Les tests doivent examiner qualité, taux de refus et erreurs préjudiciables ; la mitigation peut combiner données, consignes, contrôles et revue humaine.

12.15 Red teaming

Le red teaming cherche activement les défaillances : contournements, ambiguïtés, encodages, conversations longues, conflits d'instructions et enchaînements d'outils. Chaque incident devient idéalement un test de non-régression. Il faut préserver un jeu final caché afin de ne pas entraîner implicitement le système sur toutes les attaques connues.

12.16 Défense en profondeur

La sécurité ne repose pas sur un unique prompt. Elle combine politique, classification d'entrée et de sortie, permissions, sandbox, validation de schémas, modération, limites de débit, confirmation humaine et audit. On teste aussi le sur-refus : un système qui bloque toutes les demandes est sûr en apparence mais inutilisable.

Réflexe. Avant livraison, répondre à quatre questions : que signifie réussir ? quels échecs sont graves ? comment seront-ils détectés ? qui peut arrêter ou corriger le système ?

Chapitre 13

Déploiement, optimisation et monitoring

13.1 Du prototype au service

Un service LLM comprend généralement une API, une couche d'authentification, un orchestrateur, le modèle, des outils ou bases documentaires, un cache et une télémétrie. Les limites de temps, de coût et de concurrence doivent être explicites. La production demande de gérer les pannes partielles, pas seulement le cas nominal.

13.2 Préfill et décodage

L'inférence possède deux phases. Le *prefill* traite tous les tokens d'entrée en parallèle et remplit le cache KV. Le décodage produit ensuite les tokens un à un. Un long prompt pèse sur le temps avant le premier token ; une longue réponse pèse sur la durée totale.

On suit notamment TTFT (time to first token), TPOT (time per output token), latence totale et débit. Approximativement,

$$T_{\text{total}} \approx T_{\text{prefill}} + N_{\text{sortie}} T_{\text{token}}.$$

Les percentiles p50, p95 et p99 décrivent mieux l'expérience que la seule moyenne.

13.3 Batching continu

Regrouper plusieurs requêtes améliore l'utilisation du matériel. Le batching continu insère de nouvelles requêtes lorsque d'autres terminent, mais il faut préserver équité et objectifs de latence. La taille optimale dépend des longueurs de contexte et de sortie.

13.4 Cache KV

Les clés et valeurs d'attention des tokens déjà traités sont conservées pour éviter de les recalculer. Cette mémoire croît avec nombre de couches, longueur du contexte, taille des représentations et nombre de requêtes. Les techniques de paged attention et le partage de préfixes réduisent fragmentation et duplication.

13.5 Quantification

La quantification représente poids ou activations avec moins de bits. Pour une quantification affine,

$$q = \text{round}\left(\frac{x}{s}\right) + z, \quad \hat{x} = s(q - z),$$

où s est l'échelle et z le zéro. INT8 ou 4 bits réduisent mémoire et parfois latence, au prix d'une erreur à mesurer sur les tâches réelles. La compatibilité matérielle est déterminante.

13.6 Distillation et pruning

La distillation entraîne un petit modèle à reproduire les distributions ou réponses d'un enseignant. Une perte typique mélange vérité terrain et imitation :

$$\mathcal{L} = \alpha \mathcal{L}_{\text{task}} + (1 - \alpha) \mathcal{L}_{\text{distill}}.$$

Le pruning retire des poids, têtes ou blocs peu utiles. Ces optimisations exigent une réévaluation de qualité et de sécurité.

13.7 Décodage spéculatif

Un petit modèle propose plusieurs tokens qu'un grand modèle vérifie en parallèle. Les tokens acceptés conservent la distribution cible tout en accélérant le décodage lorsque les deux modèles sont souvent d'accord.

13.8 Choisir le modèle

Comparer qualité sur le jeu métier, contexte utile, latence, coût, confidentialité, contrôle, disponibilité et facilité d'exploitation. Un grand modèle n'est pas toujours optimal. Un routeur peut envoyer les cas simples vers un modèle économique et escalader les cas difficiles.

13.9 Cache applicatif

Le cache exact réutilise une réponse pour une requête identique. Un cache sémantique rapproche des requêtes équivalentes, avec davantage de risque. La clé doit inclure versions du modèle, du prompt, des sources et des permissions ; sinon une réponse périmée ou non autorisée peut être servie.

13.10 Coût

Un modèle simple de coût est

$$C = N_{\text{in}}c_{\text{in}} + N_{\text{out}}c_{\text{out}} + C_{\text{retrieval}} + C_{\text{outils}}.$$

Mesurer coût par tâche réussie est plus pertinent que coût par requête. Réduire les répétitions, tronquer prudemment le contexte et router les tâches produit souvent plus de gain qu'une micro-optimisation isolée.

13.11 SLO et capacité

Définir des objectifs de disponibilité, latence et qualité ainsi qu'un budget d'erreur. Prévoir pics, quotas, files d'attente et dégradation contrôlée. La loi de Little relie concurrence moyenne L , taux d'arrivée λ et temps moyen W :

$$L = \lambda W.$$

Elle aide à estimer la pression sur les ressources, sous ses hypothèses de stabilité.

13.12 Observabilité

Une trace relie requête, version du prompt, modèle, récupération, appels d'outils, tokens, latences, coûts et résultat. Les logs sont structurés et expurgés des secrets. Des tableaux de bord suivent erreurs, taux d'abstention, succès métier et dérive, avec alertes réellement actionnables.

13.13 Versionnement et livraison progressive

Versionner modèle, prompt, index, outils et politiques comme un ensemble. Tester hors ligne, puis shadow traffic, canary ou A/B test. Conserver la possibilité de revenir rapidement en arrière. Une amélioration globale ne doit pas masquer une régression sur un segment critique.

13.14 Dérive et boucle de retour

Les documents, usages et attaques changent. Surveiller distribution des requêtes, longueurs, langues, échecs, feedback et performances sur un jeu sentinelle. Les retours utilisateurs sont bruités : ils doivent être nettoyés, échantillonnés et interprétés avant de devenir données d'entraînement.

13.15 Incidents

Préparer runbooks, responsables, seuils d'arrêt et mécanismes de réduction de capacité. En cas d'incident : contenir, préserver les traces, comprendre l'étendue, corriger, tester et ajouter une non-régression. Ne jamais exposer les données sensibles dans le compte rendu.

13.16 Architecture de référence

```
Client -> API -> Auth/quotas -> Orchestrateur
                        |-> Modele / routeur
                        |-> Retrieval / outils
                        |-> Guardrails
                        '-> Traces et metriques
```

Attention. Optimiser seulement après mesure. La meilleure architecture est celle qui respecte simultanément qualité, sécurité, latence, coût et capacité d'exploitation de l'équipe.

Première partie

Mise en pratique

Chapitre 14

TP 1 – Construire un petit Transformer

14.1 Objectifs

Ce TP vise à relier les équations à une implémentation minimale : préparer un corpus, tokenizer, former des lots, appliquer attention causale, entraîner puis générer. Il ne cherche pas la performance d'un grand modèle, mais la compréhension des dimensions et des flux.

14.2 Données et protocole

Choisir un corpus libre de droits, conserver un jeu de validation distinct et documenter le nettoyage. Construire un vocabulaire de caractères ou un petit BPE. Pour une séquence $(x_1, \dots, x_T)$, la cible est $(x_2, \dots, x_{T+1})$. Comparer à un modèle n-gram afin de disposer d'une baseline.

14.3 Dimensions à vérifier

Avec un lot B , une longueur T et une largeur d , l'entrée a la forme $B \times T \times d$. Pour h têtes, chaque tête utilise généralement $d_h = d/h$. Les logits finaux ont la forme $B \times T \times |V|$. Écrire ces formes à chaque ligne évite la majorité des erreurs.

14.4 Étapes

1. Implémenter embeddings de tokens et de positions.
2. Projeter vers Q , K , V , calculer les scores et appliquer le masque triangulaire.
3. Ajouter projection de sortie, connexion résiduelle, normalisation et MLP.
4. Empiler deux à quatre blocs et terminer par la tête vocabulaire.

5. Entraîner avec cross-entropie, suivre pertes train/validation et norme du gradient.
6. Générer avec greedy, température puis top- k ; comparer les sorties.

14.5 Expériences

Faire varier longueur de contexte, nombre de têtes, largeur et dropout une variable à la fois. Mesurer paramètres, temps par lot et perplexité. Retirer le masque causal : la perte devient artificiellement bonne car le modèle voit le futur. Cette expérience illustre une fuite de cible.

14.6 Questions d'analyse

Pourquoi une faible perte ne garantit-elle pas un texte utile ? Quel est l'effet d'une température proche de zéro ? Pourquoi la génération devient-elle lente token par token ? Où intervient le cache KV ? Présenter résultats, limites et une proposition d'expérience suivante.

Chapitre 15

TP 2 – Adapter un modèle avec LoRA

15.1 Objectifs

Adapter un modèle pré-entraîné à une tâche supervisée sans modifier tous ses poids. Comparer zero-shot, prompting avec exemples et LoRA selon qualité, coût et stabilité.

15.2 Préparer les exemples

Chaque exemple comporte instruction, entrée éventuelle et sortie attendue. Dédupliquer, contrôler les licences, retirer données sensibles et inspecter les distributions de longueur. Séparer par source ou par période si des exemples proches risquent de fuiter entre train et test.

15.3 Principe

Une matrice gelée $W \in \mathbb{R}^{d_{out} \times d_{in}}$ reçoit une mise à jour de faible rang :

$$W' = W + \frac{\alpha}{r}BA, \quad A \in \mathbb{R}^{r \times d_{in}}, \quad B \in \mathbb{R}^{d_{out} \times r}.$$

On entraîne A et B , avec r petit. Calculer le nombre de paramètres appris et le comparer à $d_{out}d_{in}$.

15.4 Protocole

Fixer d'abord une baseline reproductible. Choisir modules cibles, rang, alpha, dropout, taux d'apprentissage, taille de lot effective et nombre d'époques. Sauvegarder séparément

l'adaptateur et la configuration. Évaluer régulièrement sur un jeu non vu, sans choisir le meilleur modèle sur le test final.

15.5 Analyse d'erreurs

Classer les erreurs : format, instruction ignorée, manque de connaissance, confusion d'étiquette, hallucination. Une amélioration moyenne peut cacher une dégradation des cas rares. Tester aussi des formulations différentes de celles du train.

15.6 Livrables

Fournir fiche de données, paramètres, courbes, tableau comparatif, exemples représentatifs et limites. Expliquer si l'adaptation a appris le comportement souhaité ou simplement mémorisé des formulations.

Chapitre 16

TP 3 – Construire et évaluer un RAG

16.1 Objectifs et corpus

Créer un assistant répondant uniquement à partir d'un petit corpus documentaire versionné. Chaque document porte identifiant, titre, date, droits et niveau d'accès. Construire vingt à cinquante questions, dont quelques questions sans réponse.

16.2 Pipeline minimal

Extraire le texte, découper, calculer les embeddings, indexer, retrouver les k passages, assembler le contexte et générer avec citations. Stocker pour chaque réponse les identifiants réellement fournis au modèle.

16.3 Baseline lexicale et dense

Comparer BM25, embeddings puis fusion hybride. Tester plusieurs tailles et chevauchements de chunks. Pour chaque variante, mesurer Recall@ k , MRR, latence et volume de contexte. Le meilleur retrieval n'est pas forcément celui qui transmet le plus de texte.

16.4 Jeu d'évaluation

Annoter documents pertinents, réponse de référence et possibilité d'abstention. Séparer :

- récupération : le bon passage est-il présent ?
- contexte : les passages sont-ils utiles, divers et autorisés ?
- génération : chaque affirmation est-elle soutenue ?
- expérience : réponse claire, citation accessible, latence acceptable ?

16.5 Expériences

Ajouter reranker, filtre de métadonnées et reformulation de requête. Tester une injection cachée dans un document ; vérifier qu'elle ne déclenche ni changement de rôle ni fuite. Modifier ou supprimer un document et vérifier que l'index suit la mise à jour.

16.6 Rapport

Présenter matrice d'expériences, intervalles de confiance et dix erreurs analysées. Proposer une amélioration ciblée pour chaque étage fautif. Ne pas conclure qu'il faut changer de LLM si le passage pertinent n'a jamais été récupéré.

Chapitre 17

TP 4 – Agent outillé sous contraintes

17.1 Scénario

Construire un assistant qui consulte un catalogue, calcule un devis et prépare une commande fictive. Les outils d'écriture restent simulés. Toute finalisation exige confirmation explicite et clé d'idempotence.

17.2 Contrats d'outils

Définir pour chaque outil nom, objectif, schéma JSON, erreurs possibles et permissions. Ajouter validation des bornes et rejeter les champs inconnus. Les réponses de catalogue sont des données non fiables, jamais des instructions.

17.3 Contrôleur

Limiter nombre d'étapes, durée et budget. Journaliser décisions courtes, appels, résultats et erreurs. Après timeout d'une écriture, rechercher l'identifiant avant de réessayer. Prévoir annulation et reprise.

17.4 Tests

Tester cas nominal, produit absent, prix incohérent, outil indisponible, double clic, instruction malveillante dans une description et demande dépassant les droits. Mesurer réussite, exactitude des arguments, appels inutiles, latence et violations.

17.5 Discussion

Identifier ce qui doit être déterministe et ce qui peut rester génératif. Discuter le moment où un workflow fixe est préférable à un agent autonome. Produire un diagramme des frontières de confiance.

Deuxième partie

Exercices et corrigés

Chapitre 18

Exercices

18.1 Modélisation du langage

1. Factoriser $P(x_1, \dots, x_T)$ avec la règle de chaîne.
2. Une cross-entropie moyenne vaut 2,3 nats. Calculer la perplexité.
3. Expliquer pourquoi comparer des perplexités avec deux tokenizers différents est délicat.
4. Donner un exemple de fuite entre pré-entraînement et benchmark.

18.2 Tokenisation et embeddings

1. Comparer mot, caractère et sous-mot pour une langue morphologiquement riche.
2. Pour $u = (1, 2)$ et $v = (2, 1)$, calculer la similarité cosinus.
3. Pourquoi la proximité vectorielle ne suffit-elle pas à établir une relation causale ?
4. Proposer deux métriques pour évaluer un embedding de recherche.

18.3 Attention

1. Donner les dimensions de $QK^\top$ pour $T = 128$, $d_k = 64$.
2. Expliquer le facteur $1/\sqrt{d_k}$.
3. Dessiner le masque causal pour quatre tokens.
4. Comparer coût de l'attention complète et d'une fenêtre locale de largeur w .

18.4 Entraînement et adaptation

1. Distinguer pré-entraînement, SFT et apprentissage par préférences.
2. Calculer les paramètres LoRA pour une matrice 4096×4096 et $r = 8$.

3. Pourquoi une petite base très propre peut-elle battre une grande base bruitée ?
4. Citer trois signes de surapprentissage lors d'un fine-tuning.

18.5 Génération

1. Décrire l'effet de la température sur une distribution de logits.
2. Distinguer top- k et top- p .
3. Pourquoi beam search n'est-il pas toujours idéal pour le dialogue ?
4. Concevoir une sortie JSON robuste pour trois champs obligatoires.

18.6 RAG et agents

1. Un Recall@5 faible mais une génération fidèle indique quelle panne ?
2. Comparer fine-tuning et RAG pour une réglementation changeante.
3. Pourquoi faut-il séparer lecture et écriture dans les outils ?
4. Décrire un retry sûr après timeout de paiement.

18.7 Évaluation et production

1. Concevoir un test par paires sans biais de position.
2. Distinguer factualité, fidélité et utilité.
3. Pourquoi suivre p95 plutôt que seulement la latence moyenne ?
4. Donner une clé correcte pour un cache de réponse RAG.

Chapitre 19

Corrigés

19.1 Modélisation du langage

La règle de chaîne donne $P(x_{1:T}) = \prod_{t=1}^T P(x_t | x_{<t})$. La perplexité vaut $e^{2.3} \simeq 9,97$. Des tokenizers différents changent nombre et difficulté des unités ; la perte par token n'est donc pas directement comparable. Une fuite existe si des questions du benchmark ou leurs corrigés figurent dans le corpus d'entraînement.

19.2 Tokenisation et embeddings

Les mots rendent le vocabulaire énorme et gèrent mal l'inconnu ; les caractères allongent beaucoup les séquences ; les sous-mots offrent un compromis. Le cosinus vaut $4/(\sqrt{5}\sqrt{5}) = 0,8$. Une corrélation géométrique apprise reflète les données et ne démontre aucun mécanisme causal. Pour la recherche : Recall@k, MRR ou nDCG, complétés par une évaluation humaine.

19.3 Attention

$QK^\top$ a la dimension 128×128 par tête. La division stabilise la variance des produits scalaires et évite des softmax trop saturés. Le masque autorise les positions (i, j) telles que $j \leq i$. L'attention complète coûte $O(T^2d)$; une fenêtre bornée coûte environ $O(Twd)$.

19.4 Entraînement et adaptation

Le pré-entraînement acquiert des régularités générales, le SFT enseigne des réponses attendues, les préférences favorisent certaines sorties. LoRA apprend $r(d_{in} + d_{out}) = 8(4096 + 4096) = 65\,536$ paramètres au lieu de 16 777 216. Des données propres donnent un signal cohérent. Surapprentissage : perte train qui baisse tandis que validation remonte,

mémorisation d'exemples, chute hors distribution.

19.5 Génération

Une petite température concentre la masse ; une grande l'aplatit. Top- k conserve un nombre fixe de tokens, top- p le plus petit ensemble dépassant une masse cumulée. Beam search favorise parfois des réponses génériques et répétitives. Un schéma JSON impose types, champs requis, valeurs autorisées et refus des propriétés supplémentaires.

19.6 RAG et agents

Le retrieval échoue avant même la génération. Une règle changeante se prête au RAG, qui met à jour les sources, tandis que le fine-tuning convient mieux au comportement ou au style. Séparer lecture et écriture applique le moindre privilège. Après timeout, rechercher d'abord l'opération avec une clé d'idempotence ; ne répéter que si son absence est établie.

19.7 Évaluation et production

Alterner aléatoirement gauche/droite et masquer l'identité des systèmes. La factualité compare au monde, la fidélité aux sources fournies, l'utilité au besoin. Le p95 révèle l'expérience des requêtes lentes. Une clé de cache inclut requête normalisée, identité/droits, versions du modèle, prompt, index et documents.

Troisième partie

Révision et références

Chapitre 20

Aide-mémoire mathématique

20.1 Probabilités et information

$$P(x_{1:T}) = \prod_t P(x_t \mid x_{<t}), \quad H(p, q) = - \sum_x p(x) \log q(x), \quad \text{PPL} = e^H.$$
$$D_{KL}(p \parallel q) = \sum_x p(x) \log \frac{p(x)}{q(x)}, \quad I(X; Y) = D_{KL}(p(x, y) \parallel p(x)p(y)).$$

20.2 Algèbre et attention

$$\cos(u, v) = \frac{u^\top v}{\|u\| \|v\|}, \quad \text{Att}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V.$$

Pour h têtes : concaténer $\text{Att}_1, \dots, \text{Att}_h$ puis projeter. Le masque causal $M_{ij} = -\infty$ lorsque $j > i$.

20.3 Optimisation

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t), \quad \text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

Avec température τ : $p_i \propto e^{z_i/\tau}$. Une norme de gradient anormalement élevée signale souvent instabilité ou lot atypique.

20.4 Évaluation

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F_\beta = (1 + \beta^2) \frac{PR}{\beta^2 P + R}.$$

Toujours préciser unité d'analyse, agrégation micro/macro, seuil et intervalle d'incertitude.

Chapitre 21

Glossaire

Alignement Ajustement du comportement aux intentions et contraintes humaines.

Attention Pondération dynamique des représentations utilisées pour produire une nouvelle représentation.

BPE Tokenisation qui fusionne itérativement des unités fréquentes.

Chunk Fragment documentaire indexé dans un RAG.

Contexte Tokens accessibles au modèle pour la prédiction courante.

Embedding Vecteur dense représentant une unité ou un document.

Few-shot Prompt contenant quelques démonstrations.

Fine-tuning Poursuite de l'entraînement sur des données ciblées.

Grounding Ancrage d'une réponse dans des preuves ou données identifiées.

Hallucination Affirmation non soutenue ou incorrecte produite par le système.

Instruction tuning SFT sur des paires instruction-réponse.

KV cache Mémoire des clés et valeurs déjà calculées pendant le décodage.

LoRA Adaptation par mises à jour matricielles de faible rang.

Perplexité Exponentielle de la cross-entropie moyenne par token.

Prompt injection Entrée hostile visant à détourner les instructions ou outils.

Quantification Représentation numérique à précision réduite.

RAG Génération augmentée par récupération documentaire.

Reranker Modèle réordonnant les documents candidats.

SFT Fine-tuning supervisé.

Token Unité discrète traitée par le modèle.

Transformer Architecture fondée principalement sur attention et blocs feed-forward.

TTFT Durée entre requête et premier token produit.

Zero-shot Résolution sans exemple démonstratif dans le prompt.

Chapitre 22

Fiches de révision

Fiche 1 – Modèles de langage

Idée centrale. Un modèle de langage estime une distribution conditionnelle sur le prochain token et factorise une séquence par la règle de chaîne.

À savoir expliquer. N-gram, cross-entropie, perplexité, généralisation, séparation entraînement/validation/test.

Piège classique. Confondre faible perplexité, vérité factuelle et utilité.

Auto-test.

1. Donner les définitions sans notes.
2. Refaire les équations essentielles et indiquer les dimensions.
3. Expliquer un échec concret et choisir une mesure adaptée.
4. Relier ce chapitre au précédent et au suivant.

Fiche 2 – Tokens et vecteurs

Idée centrale. Le tokenizer définit les unités et les embeddings les projettent dans un espace continu appris.

À savoir expliquer. BPE, unigram, vocabulaire, OOV, similarité cosinus, positions.

Piège classique. Comparer naïvement les longueurs ou perplexités de tokenizers différents.

Auto-test.

1. Donner les définitions sans notes.
2. Refaire les équations essentielles et indiquer les dimensions.
3. Expliquer un échec concret et choisir une mesure adaptée.
4. Relier ce chapitre au précédent et au suivant.

Fiche 3 – RNN et LSTM

Idée centrale. Les réseaux récurrents résument le passé dans un état ; les portes des LSTM facilitent la circulation du gradient.

À savoir expliquer. BPTT, gradients évanescents/explosifs, cellules et portes, limites séquentielles.

Piège classique. Croire que la mémoire théorique garantit la conservation d’une information longue.

Auto-test.

1. Donner les définitions sans notes.
2. Refaire les équations essentielles et indiquer les dimensions.
3. Expliquer un échec concret et choisir une mesure adaptée.
4. Relier ce chapitre au précédent et au suivant.

Fiche 4 – Transformer

Idée centrale. L'attention met directement en relation les positions, puis résidus, normalisations et MLP transforment les représentations.

À savoir expliquer. Q/K/V, softmax, masque causal, multi-têtes, complexité quadratique.

Piège classique. Oublier le masque ou les dimensions ; interpréter une carte d'attention comme explication causale.

Auto-test.

1. Donner les définitions sans notes.
2. Refaire les équations essentielles et indiquer les dimensions.
3. Expliquer un échec concret et choisir une mesure adaptée.
4. Relier ce chapitre au précédent et au suivant.

Fiche 5 – Pré-entraînement

Idée centrale. Un objectif auto-supervisé transforme un immense corpus en signal prédictif.

À savoir expliquer. Causal LM, masked LM, lois d'échelle, qualité/déduplication, contamination.

Piège classique. Attribuer toutes les capacités à la taille en ignorant données et calcul.

Auto-test.

1. Donner les définitions sans notes.
2. Refaire les équations essentielles et indiquer les dimensions.
3. Expliquer un échec concret et choisir une mesure adaptée.
4. Relier ce chapitre au précédent et au suivant.

Fiche 6 – Familles de modèles

Idée centrale. Encodeur, décodeur et encodeur-décodeur imposent des masques et usages différents.

À savoir expliquer. BERT, GPT, T5, représentations bidirectionnelles, génération autorégressive.

Piège classique. Choisir un modèle uniquement par nombre de paramètres.

Auto-test.

1. Donner les définitions sans notes.
2. Refaire les équations essentielles et indiquer les dimensions.
3. Expliquer un échec concret et choisir une mesure adaptée.
4. Relier ce chapitre au précédent et au suivant.

Fiche 7 – Génération

Idée centrale. Le modèle produit des probabilités ; l'algorithme de décodage détermine la trajectoire réellement observée.

À savoir expliquer. Greedy, beam, température, top-k, top-p, répétition, cache KV.

Piège classique. Chercher un réglage universel sans jeu d'évaluation métier.

Auto-test.

1. Donner les définitions sans notes.
2. Refaire les équations essentielles et indiquer les dimensions.
3. Expliquer un échec concret et choisir une mesure adaptée.
4. Relier ce chapitre au précédent et au suivant.

Fiche 8 – Prompting

Idée centrale. Un bon prompt spécifie tâche, contexte, contraintes et format ; le code valide la sortie.

À savoir expliquer. Zero/few-shot, décomposition, JSON Schema, défense contre entrées hostiles.

Piège classique. Utiliser le prompt comme seule barrière de sécurité.

Auto-test.

1. Donner les définitions sans notes.
2. Refaire les équations essentielles et indiquer les dimensions.
3. Expliquer un échec concret et choisir une mesure adaptée.
4. Relier ce chapitre au précédent et au suivant.

Fiche 9 – Adaptation

Idée centrale. SFT et préférences modifient le comportement ; PEFT réduit les paramètres entraînés.

À savoir expliquer. Instruction tuning, LoRA, rang, données, surapprentissage, oubli catastrophique.

Piège classique. Fine-tuner pour injecter des faits qui changent fréquemment.

Auto-test.

1. Donner les définitions sans notes.
2. Refaire les équations essentielles et indiquer les dimensions.
3. Expliquer un échec concret et choisir une mesure adaptée.
4. Relier ce chapitre au précédent et au suivant.

Fiche 10 – RAG

Idée centrale. Le RAG récupère des preuves puis conditionne la génération ; chaque étage s'évalue séparément.

À savoir expliquer. Chunking, BM25, embeddings, ANN, hybride, reranking, Recall@k, fidélité.

Piège classique. Accuser la génération lorsque le bon document n'était pas dans le contexte.

Auto-test.

1. Donner les définitions sans notes.
2. Refaire les équations essentielles et indiquer les dimensions.
3. Expliquer un échec concret et choisir une mesure adaptée.
4. Relier ce chapitre au précédent et au suivant.

Fiche 11 – Agents

Idée centrale. Un agent combine modèle, état, outils et contrôleur dans une boucle bornée.

À savoir expliquer. Schémas, permissions, idempotence, mémoire, arrêt, human-in-the-loop, traces.

Piège classique. Confondre texte provenant d'un outil avec une instruction autorisée.

Auto-test.

1. Donner les définitions sans notes.
2. Refaire les équations essentielles et indiquer les dimensions.
3. Expliquer un échec concret et choisir une mesure adaptée.
4. Relier ce chapitre au précédent et au suivant.

Fiche 12 – Évaluation et sécurité

Idée centrale. L'évaluation doit refléter usages et risques ; la sécurité repose sur plusieurs contrôles indépendants.

À savoir expliquer. Jeux stratifiés, comparaisons, juge LLM, factualité, injection, red team, abstention.

Piège classique. Optimiser un score moyen et ignorer les cas critiques ou le sur-refus.

Auto-test.

1. Donner les définitions sans notes.
2. Refaire les équations essentielles et indiquer les dimensions.
3. Expliquer un échec concret et choisir une mesure adaptée.
4. Relier ce chapitre au précédent et au suivant.

Fiche 13 – Production

Idée centrale. Le déploiement équilibre qualité, latence, coût, capacité, confidentialité et opérabilité.

À savoir expliquer. Préfill/decode, batching, KV cache, quantification, SLO, canary, dérive, incidents.

Piège classique. Optimiser avant de mesurer ou changer plusieurs versions sans traçabilité.

Auto-test.

1. Donner les définitions sans notes.
2. Refaire les équations essentielles et indiquer les dimensions.
3. Expliquer un échec concret et choisir une mesure adaptée.
4. Relier ce chapitre au précédent et au suivant.

Chapitre 23

Études de cas transversales

23.1 Assistant documentaire interne

L'entreprise veut répondre à partir de procédures dont les droits diffèrent par équipe. Une solution raisonnable combine authentification, filtres d'accès avant retrieval, citations, abstention et journalisation expurgée. Le jeu de test contient questions répondables, obsolètes, contradictoires et interdites. Les métriques distinguent Recall@ k , fidélité, exactitude métier et fuite d'information. Un déploiement canary commence sur un petit groupe ; chaque réponse permet de signaler document manquant ou citation incorrecte.

23.2 Extraction de contrats

La sortie est un JSON contenant parties, dates, montants et clauses. On préfère schéma strict, validation, citations vers les pages et revue humaine pour les champs à fort enjeu. Les données sont séparées par contrat, pas par page, afin d'éviter la fuite. Exact match convient aux dates normalisées ; précision/rappel par champ convient aux clauses. Le système ne doit pas inventer un champ absent : il produit une valeur nulle et une justification localisable.

23.3 Support client outillé

Le modèle consulte statut de commande et politique de retour ; le remboursement reste une action confirmée. Les réponses d'outil sont validées et les identifiants appartiennent à l'utilisateur authentifié. Une clé d'idempotence protège les écritures. On mesure résolution au premier contact, exactitude, escalades, coût, p95 et incidents. Les conversations insatisfaisantes alimentent un processus d'analyse, pas directement l'entraînement.

23.4 Synthèse médicale assistée

Le système résume des notes pour un professionnel sans établir seul un diagnostic. Provenance, confidentialité, complétude et traçabilité dominant. On évalue omissions critiques et contradictions, avec spécialistes et scénarios sentinelles. Les affirmations importantes renvoient au document source. Toute incertitude est visible ; l'humain reste responsable de la décision.

23.5 Assistant de programmation

La qualité ne se limite pas à la compilation : tests, sécurité, maintenabilité et respect du dépôt comptent. Exécuter dans un environnement isolé, limiter réseau et secrets, inspecter les changements, puis lancer tests ciblés. Un benchmark public donne un signal, mais un jeu de tâches issues du code réel mesure mieux la valeur. Les échecs sont classés entre compréhension, modification, dépendance, test et environnement.

23.6 Choisir l'architecture

Pour chaque cas, partir du besoin et des risques. Utiliser prompting si la capacité existe déjà, RAG pour connaissances externes changeantes, fine-tuning pour comportement récurrent, outils pour actions ou calculs exacts, agent seulement pour décisions séquentielles réellement variables. Cette progression limite complexité et surface d'erreur.

Chapitre 24

Bibliographie commentée

Bibliographie

- [1] A. Vaswani et al., *Attention Is All You Need*, 2017. Article fondateur du Transformer.
- [2] J. Devlin et al., *BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding*, 2018.
- [3] T. Brown et al., *Language Models are Few-Shot Learners*, 2020.
- [4] C. Raffel et al., *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*, JMLR, 2020.
- [5] J. Kaplan et al., *Scaling Laws for Neural Language Models*, 2020.
- [6] J. Hoffmann et al., *Training Compute-Optimal Large Language Models*, 2022.
- [7] L. Ouyang et al., *Training Language Models to Follow Instructions with Human Feedback*, 2022.
- [8] E. Hu et al., *LoRA : Low-Rank Adaptation of Large Language Models*, 2021.
- [9] P. Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, 2020.
- [10] V. Karpukhin et al., *Dense Passage Retrieval for Open-Domain Question Answering*, 2020.
- [11] S. Yao et al., *ReAct : Synergizing Reasoning and Acting in Language Models*, 2022.
- [12] S. Lin, J. Hilton, O. Evans, *TruthfulQA*, 2021.
- [13] P. Liang et al., *Holistic Evaluation of Language Models*, 2022.
- [14] T. Dao et al., *FlashAttention*, 2022.
- [15] Y. Leviathan et al., *Fast Inference from Transformers via Speculative Decoding*, 2023.

Conclusion générale

Un LLM est d'abord un modèle probabiliste de séquences, devenu puissant grâce aux représentations, à l'attention, aux données et au calcul. Un système utile ajoute autour de lui contexte, retrieval, outils, validations, évaluations et opérations. Comprendre cette chaîne évite deux erreurs symétriques : attribuer au modèle des pouvoirs qu'il n'a pas, ou sous-estimer ce qu'une architecture bien conçue peut accomplir.

La démarche recommandée reste expérimentale : définir le succès, établir une baseline, isoler les étages, mesurer, analyser les erreurs et complexifier seulement si le gain est démontré. Les modèles évolueront ; ces réflexes resteront valables.